crack me 2

- First after i set the crackme file in my directory and i give it a permission to run and identify its file type

```
4197  file crackme2
4198  chmod +x crackme2
4199  ./crackme2
```

```
ls -lh
otal 112K
-rwxrwxr-x 1 jovani jovani 109K Sep  5 04:19 crackme2
```

With a total permission 771

And then i try to use strings fro static analysis



I can't find any useful information so i switched for dynamic analysis using #gdb

---

GDB session

first step we analyze the decompiled c++ source code of the main function. By inspecting the logic, we uncovered 3 rules how the binary operates:

1 input Sanitization

- removes spaces

2 Length guard

- It tells the username is shorter than 4 characters and prints Too short and exits early.
- Prints `Too short`

3 The Key Generator

- Instead of hard coded master code the program(app) runs a function

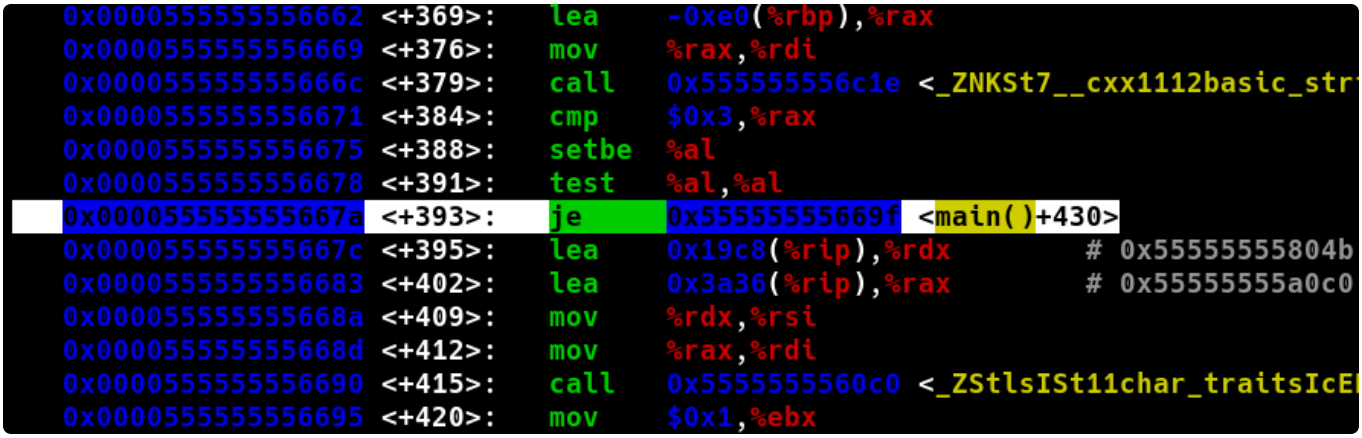
```
    generateSerial(&local_78, &username);
```

And we loaded the binary to #gdb using
`gdb ./crackme2`

and ran

```
(gdb) disassemble main
```

And then we looked the branching logic controlling the output. we found a #je (jump if Equal) instruction pointing the address `0x55555555669f`.



which exactly led to the failure flag `[-] Access Denied!`

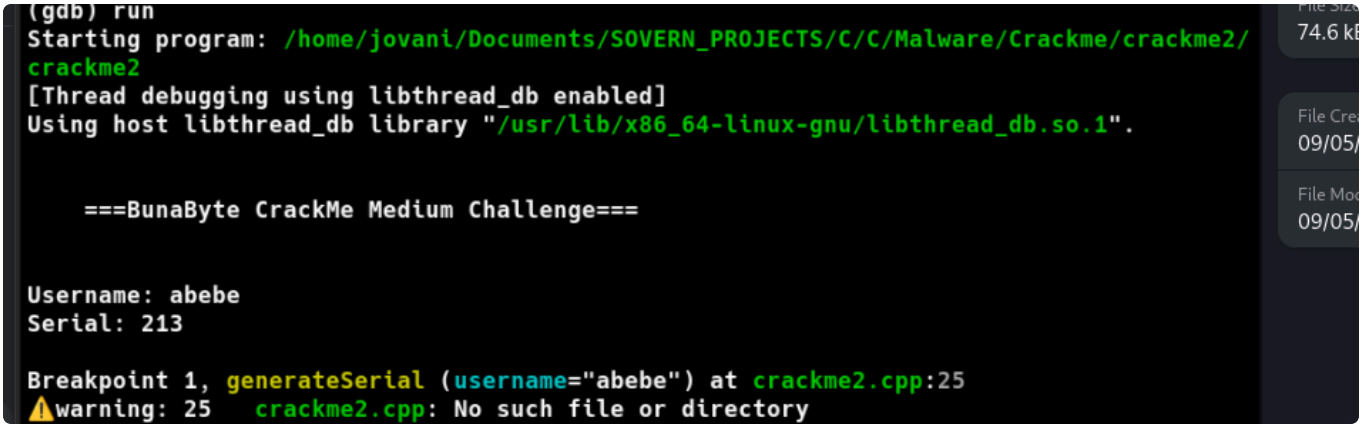
Because the binary was compiled as a #PIE (Position Independent Executable), Memory addresses shifted dynamically during execution.
So,instead of chasing fluctuating instruction pointers to patch the check, we opted for an easier, method `Intercepting the return value` As follows

```
(gdb) break generateSerial
```

Then after this we initialized the excution flow

```
(gdb) run
```

and provide our baseline inputs:

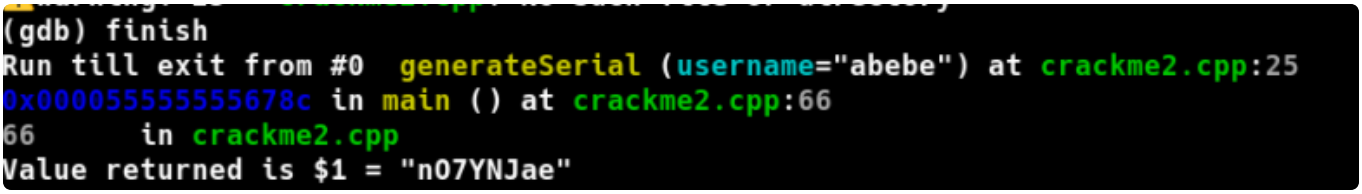


As soon as the binary hit the key generator, GDB paused execution. We then issued the

```
(gdb) Finish
```

This instructed GDB to execute the rest of the `generateSerial` function immediately and pause the exact moment it returned to `main`. Because it paused on the return, GDB captured the function's output directly from the CPU registers, printing:

```
value returned is $1 = "n07YNJae"
```



Finally we clear all the Break points and restart and then we get the flag

```
[+] Access Granted!
Flag: bb{medium_crackme_solved}
```